



Bikkad IT Institute

NumPy Tricks Notes

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

NumPy Tricks Notes

Sorting, Searching, Set Ops and Useful Functions

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment: Jupyter Notebook or Google Colab

Topic Index

1. np.sort
2. np.append
3. np.concatenate
4. np.unique
5. np.expand_dims
6. np.where
7. np.argmax and np.argmin
8. np.cumsum and np.cumprod
9. np.percentile and np.median
10. np.histogram
11. np.corrcoef
12. np.isin
13. np.flip
14. np.put and np.delete
15. Set functions
16. np.clip

1. np.sort

Returns a sorted copy of an array, without changing the original.

Syntax

```
np.sort(a, axis=-1)
```

Example 1: Ascending sort of a 1D array

```
import numpy as np

np.random.seed(1)
a = np.random.randint(1, 100, 8)
print("Original array:", a)

sorted_a = np.sort(a)
print("Sorted array:", sorted_a)
```

Output / Result

```
Original array: [38 13 73 10 76  6 80 65]
Sorted array: [ 6 10 13 38 65 73 76 80]
```

Explanation

np.sort() returns a new array with the values arranged from smallest to largest by default; the original array a is left unchanged.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 2: Descending sort

```
import numpy as np

np.random.seed(1)
a = np.random.randint(1, 100, 8)
print("Original array:", a)

ascending = np.sort(a)
print("Ascending sorted:", ascending)

descending = ascending[::-1]
print("Descending sorted:", descending)
```

Output / Result

```
Original array: [38 13 73 10 76  6 80 65]
Ascending sorted: [ 6 10 13 38 65 73 76 80]
Descending sorted: [80 76 73 65 38 13 10  6]
```

Explanation

NumPy has no direct 'descending' argument, so the usual trick is to sort ascending and then reverse the result with `::-1`.

Example 3: Sorting a 2D array along an axis

```
import numpy as np

np.random.seed(1)
a = np.random.randint(1, 100, 8) # same draws as Example 1
print("1D array (for context):", a)

b = np.random.randint(1, 100, 12).reshape(3, 4)
print("Original 2D array:\n", b)

sorted_axis0 = np.sort(b, axis=0)
print("Sorted along axis=0 (each column):\n", sorted_axis0)
```

Output / Result

```
1D array (for context): [38 13 73 10 76  6 80 65]
Original 2D array:
[[17  2 77 72]
 [ 7 26 51 21]
 [19 85 12 29]]
Sorted along axis=0 (each column):
[[ 7  2 12 21]
 [17 26 51 29]
 [19 85 77 72]]
```

Explanation

`axis=0` sorts each column independently (down the rows); `axis=1` (the default for 2D arrays) would sort each row independently instead.

2. np.append

Appends values along the given axis at the end of an array, returning a new array.

Syntax

```
np.append(array, values, axis=None)
```

Example 1: Appending a single value

```
import numpy as np

a = np.array([10, 20, 30])
print("Original array:", a)

appended = np.append(a, 200)
print("After append:", appended)
```

Output / Result

```
Original array: [10 20 30]
After append: [ 10  20  30 200]
```

Explanation

Without an axis, the array is treated as flat and the new value is simply added at the end.

Example 2: Appending a new column to a 2D array

```
import numpy as np

np.random.seed(2)
b = np.arange(8).reshape(4, 2)
print("Original 2D array:\n", b)

extra_col = np.random.randint(1, 10, (b.shape[0], 1))
print("New column to append:\n", extra_col)

with_new_col = np.append(b, extra_col, axis=1)
print("After appending column (axis=1):\n", with_new_col)
```

Output / Result

```
Original 2D array:
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
New column to append:
[[9]
 [9]
 [7]
 [3]]
After appending column (axis=1):
[[0 1 9]
 [2 3 9]
 [4 5 7]
 [6 7 3]]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Explanation

axis=1 appends the new values as an additional column, so the appended array must have the same number of rows as b.

Example 3: Appending a new row to a 2D array

```
import numpy as np

np.random.seed(2)
b = np.arange(8).reshape(4, 2)
print("Original 2D array:\n", b)

new_row = np.array([[99, 99]])
print("New row to append:\n", new_row)

with_new_row = np.append(b, new_row, axis=0)
print("After appending row (axis=0):\n", with_new_row)
```

Output / Result

```
Original 2D array:
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
New row to append:
[[99 99]]
After appending row (axis=0):
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [99 99]]
```

Explanation

axis=0 appends the new values as an additional row, so the appended array must have the same number of columns as b.

3. np.concatenate

Joins a sequence of arrays along an existing axis.

Syntax

```
np.concatenate((a, b), axis=0)
```

Example 1: Concatenating along axis 0 (rows)

```
import numpy as np

c = np.arange(6).reshape(2, 3)
print("Array c:\n", c)

d = np.arange(6, 12).reshape(2, 3)
print("Array d:\n", d)

stacked_rows = np.concatenate((c, d), axis=0)
print("Concatenated along axis=0 (rows):\n", stacked_rows)
```

Output / Result

```
Array c:
[[0 1 2]
 [3 4 5]]
Array d:
[[ 6  7  8]
 [ 9 10 11]]
Concatenated along axis=0 (rows):
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

Explanation

axis=0 stacks the arrays vertically, one below the other; both arrays must have the same number of columns.

Example 2: Concatenating along axis 1 (columns)

```
import numpy as np

c = np.arange(6).reshape(2, 3)
print("Array c:\n", c)

d = np.arange(6, 12).reshape(2, 3)
print("Array d:\n", d)

stacked_cols = np.concatenate((c, d), axis=1)
print("Concatenated along axis=1 (columns):\n", stacked_cols)
```

Output / Result

```
Array c:
[[0 1 2]
 [3 4 5]]
Array d:
[[ 6  7  8]
 [ 9 10 11]]
Concatenated along axis=1 (columns):
[[ 0  1  2  6  7  8]
 [ 3  4  5  9 10 11]]
```

Explanation

axis=1 joins the arrays side by side; both arrays must have the same number of rows.

OUR LOCATIONS

Example 3: Concatenating flat 1D arrays

```
import numpy as np

e1 = np.array([1, 2, 3])
print("Array e1:", e1)

e2 = np.array([4, 5])
print("Array e2:", e2)

combined = np.concatenate((e1, e2))
print("Concatenated 1D arrays:", combined)
```

Output / Result

```
Array e1: [1 2 3]
Array e2: [4 5]
Concatenated 1D arrays: [1 2 3 4 5]
```

Explanation

For 1D arrays there is only one axis, so concatenate simply chains the values end to end.

4. np.unique

Finds the distinct (unique) values in an array, sorted in ascending order.

Syntax

```
np.unique(a, return_counts=False, return_index=False)
```

Example 1: Basic unique values

```
import numpy as np

arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print("Original array:", arr)

unique_vals = np.unique(arr)
print("Unique values:", unique_vals)
```

Output / Result

```
Original array: [1 1 2 2 3 3 4 4 5]
Unique values: [1 2 3 4 5]
```

Explanation

Duplicate values are collapsed and the remaining distinct values are returned in sorted order.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 2: Counting how often each value occurs

```
import numpy as np

arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print("Original array:", arr)

values, counts = np.unique(arr, return_counts=True)
print("Unique values:", values)
print("Counts:", counts)
```

Output / Result

```
Original array: [1 1 2 2 3 3 4 4 5]
Unique values: [1 2 3 4 5]
Counts: [2 2 2 2 1]
```

Explanation

`return_counts=True` additionally returns how many times each unique value appeared in the original array.

Example 3: Locating the first occurrence of each value

```
import numpy as np

arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print("Original array:", arr)

values, idx = np.unique(arr, return_index=True)
print("Unique values:", values)
print("First-occurrence indices:", idx)
```

Output / Result

```
Original array: [1 1 2 2 3 3 4 4 5]
Unique values: [1 2 3 4 5]
First-occurrence indices: [0 2 4 6 8]
```

Explanation

`return_index=True` returns the position in the original array where each unique value first appears.

5. np.expand_dims

Inserts a new axis of size 1 at the given position, increasing an array's dimension count.

Syntax

```
np.expand_dims(a, axis)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 1: Expanding a 1D array along axis 0

```
import numpy as np

a = np.array([1, 2, 3, 4])
print("Original array:", a)
print("Original shape:", a.shape)

expanded_axis0 = np.expand_dims(a, axis=0)
print("Expanded array (axis=0):\n", expanded_axis0)
print("New shape:", expanded_axis0.shape)
```

Output / Result

```
Original array: [1 2 3 4]
Original shape: (4,)
Expanded array (axis=0):
[[1 2 3 4]]
New shape: (1, 4)
```

Explanation

axis=0 turns a flat (4,) array into a (1,4) row vector by adding a new leading dimension.

Example 2: Expanding along axis 1

```
import numpy as np

a = np.array([1, 2, 3, 4])
print("Original array:", a)

expanded = np.expand_dims(a, axis=1)
print("Expanded array (axis=1):\n", expanded)
print("New shape:", expanded.shape)
```

Output / Result

```
Original array: [1 2 3 4]
Expanded array (axis=1):
[[1]
 [2]
 [3]
 [4]]
New shape: (4, 1)
```

Explanation

axis=1 instead turns the same array into a (4,1) column vector, adding the new dimension at the end.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 3: Comparing both against the original

```
import numpy as np

a = np.array([1, 2, 3, 4])
print("Original shape:", a.shape)

shape_axis0 = np.expand_dims(a, axis=0).shape
print("expand_dims(a, axis=0) shape:", shape_axis0)

shape_axis1 = np.expand_dims(a, axis=1).shape
print("expand_dims(a, axis=1) shape:", shape_axis1)
```

Output / Result

```
Original shape: (4,)
expand_dims(a, axis=0) shape: (1, 4)
expand_dims(a, axis=1) shape: (4, 1)
```

Explanation

`expand_dims` is commonly used to reshape a 1D array into a row or column vector before a broadcasting operation that needs an explicit 2D shape.

6. np.where

Returns the indices where a condition holds, or builds a new array by choosing between two values based on that condition.

Syntax

```
np.where(condition)
np.where(condition, value_if_true, value_if_false)
```

Example 1: Finding indices where a condition holds

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

condition = a > 50
print("Condition (a > 50):", condition)

indices = np.where(condition)
print("Indices where condition holds:", indices)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Condition (a > 50): [False False  True  True False False False  True False False]
Indices where condition holds: (array([2, 3, 7]),)
```

OUR LOCATIONS

Explanation

With a single argument, `np.where(condition)` returns a tuple of index arrays for every position where the condition is True.

Example 2: Replacing values above a threshold

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

condition = a > 50
print("Condition (a > 50):", condition)

replaced = np.where(condition, 0, a)
print("Values above 50 replaced with 0:", replaced)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Condition (a > 50): [False False  True  True False False False  True False False]
Values above 50 replaced with 0: [25  4  0  0  1 22 20  0 42 11]
```

Explanation

With three arguments, `np.where` rebuilds the array: wherever the condition is True it uses the second argument (0), otherwise it keeps the original value.

Example 3: Replacing every even value

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

is_even = a % 2 == 0
print("Is-even mask:", is_even)

replaced = np.where(is_even, 0, a)
print("Even values replaced with 0:", replaced)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Is-even mask: [False  True False False False  True  True False  True False]
Even values replaced with 0: [25  0 57 73  1  0  0 75  0 11]
```

Explanation

The condition can be any Boolean expression - here every even number is replaced with 0 while odd numbers are left untouched.

7. np.argmax and np.argmin

Return the index (position) of the maximum or minimum element, rather than the value itself.

OUR LOCATIONS

Syntax

```
np.argmax(a, axis=None)
np.argmin(a, axis=None)
```

Example 1: Position of the largest and smallest value

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

max_index = np.argmax(a)
print("Index of maximum value:", max_index)

min_index = np.argmin(a)
print("Index of minimum value:", min_index)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Index of maximum value: 7
Index of minimum value: 4
```

Explanation

argmax and argmin return the position of the extreme value, which is often more useful than the value itself when you need to know where it occurred.

Example 2: argmax along columns of a 2D array

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("1D array (for context):", a)

b = np.random.randint(1, 100, 12).reshape(3, 4)
print("2D array:\n", b)

col_argmax = np.argmax(b, axis=0)
print("Index of max value in each column (axis=0):", col_argmax)
```

Output / Result

```
1D array (for context): [25  4 57 73  1 22 20 75 42 11]
2D array:
[[22 39 97 21]
 [45 94 40 15]
 [27 82 91 23]]
Index of max value in each column (axis=0): [1 1 0 2]
```

Explanation

axis=0 returns, for each column, the row index of that column's largest value.

OUR LOCATIONS

Example 3: argmax along rows of a 2D array

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
b = np.random.randint(1, 100, 12).reshape(3, 4)
print("2D array:\n", b)

row_argmax = np.argmax(b, axis=1)
print("Index of max value in each row (axis=1):", row_argmax)
```

Output / Result

```
2D array:
[[22 39 97 21]
 [45 94 40 15]
 [27 82 91 23]]
Index of max value in each row (axis=1): [2 1 2]
```

Explanation

axis=1 instead returns, for each row, the column index of that row's largest value.

8. np.cumsum and np.cumprod

Compute a running (cumulative) sum or product of array elements, rather than a single total.

Syntax

```
np.cumsum(a, axis=None)
np.cumprod(a, axis=None)
```

Example 1: Cumulative sum of a 1D array

```
import numpy as np

a = np.array([5, 10, 15, 20])
print("Original array:", a)

running_total = np.cumsum(a)
print("Cumulative sum:", running_total)
```

Output / Result

```
Original array: [ 5 10 15 20]
Cumulative sum: [ 5 15 30 50]
```

Explanation

Each output position holds the running total of all elements up to and including that position.

OUR LOCATIONS

Example 2: Cumulative sum along an axis of a 2D array

```
import numpy as np

b = np.arange(1, 13).reshape(3, 4)
print("Original 2D array:\n", b)

running_total = np.cumsum(b, axis=1)
print("Cumulative sum along axis=1 (each row):\n", running_total)
```

Output / Result

```
Original 2D array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Cumulative sum along axis=1 (each row):
[[ 1  3  6 10]
 [ 5 11 18 26]
 [ 9 19 30 42]]
```

Explanation

axis=1 restarts the running total at the beginning of each row; omitting axis would flatten the array first and cumulate over all elements.

Example 3: Cumulative product

```
import numpy as np

a = np.array([5, 10, 15, 20])
print("Original array:", a)

running_product = np.cumprod(a)
print("Cumulative product:", running_product)
```

Output / Result

```
Original array: [ 5 10 15 20]
Cumulative product: [  5  50  750 15000]
```

Explanation

np.cumprod works exactly like cumsum but multiplies the running total instead of adding to it.

9. np.percentile and np.median

Summarize the relative position of values in a dataset; the median is simply the 50th percentile.

Syntax

```
np.percentile(a, q)
np.median(a)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 1: The 50th percentile

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

p50 = np.percentile(a, 50)
print("50th percentile:", p50)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
50th percentile: 23.5
```

Explanation

The 50th percentile is the value below which half the data falls, exactly matching the definition of the median.

Example 2: Confirming it matches np.median

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

median_val = np.median(a)
print("Median:", median_val)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Median: 23.5
```

Explanation

`np.median(a)` is a convenience shortcut that always returns `np.percentile(a, 50)`.

Example 3: Multiple percentiles at once (quartiles)

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

quartiles = np.percentile(a, [25, 50, 75])
print("Quartiles (25th, 50th, 75th):", quartiles)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Quartiles (25th, 50th, 75th): [13.25 23.5  53.25]
```

OUR LOCATIONS

Explanation

Passing a list of percentages computes several percentiles in a single call, commonly used to get the quartiles of a dataset.

10. np.histogram

Counts how many values fall into each of a set of bins, the numeric basis behind a histogram chart.

Syntax

```
np.histogram(a, bins)
```

Example 1: Two wide bins

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

counts, edges = np.histogram(a, bins=[0, 50, 100])
print("Counts per bin:", counts)
print("Bin edges:", edges)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Counts per bin: [7 3]
Bin edges: [  0  50 100]
```

Explanation

The bins list defines the boundaries; counts reports how many values fall inside each interval, here below and above 50.

Example 2: More, evenly-sized bins

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

counts, edges = np.histogram(a, bins=[0, 25, 50, 75, 100])
print("Counts per bin:", counts)
print("Bin edges:", edges)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Counts per bin: [5 2 2 1]
Bin edges: [  0  25  50  75 100]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Explanation

More boundary points create a finer-grained frequency breakdown, which is exactly what a histogram plot visualizes as bars.

Example 3: Automatic equal-width bins

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

counts, edges = np.histogram(a, bins=4)
print("Counts per bin:", counts)

rounded_edges = np.round(edges, 1)
print("Bin edges (rounded):", rounded_edges)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Counts per bin: [3 3 1 3]
Bin edges (rounded): [ 1.  19.5 38.  56.5 75. ]
```

Explanation

Passing a single integer instead of a list asks NumPy to divide the data's range into that many equal-width bins automatically.

11. np.corrcoef

Returns the Pearson correlation coefficient matrix, measuring how strongly two variables move together.

Syntax

```
np.corrcoef(x, y)
```

Example 1: A strong positive correlation

```
import numpy as np

salary = np.array([20000, 30000, 40000, 50000, 60000])
print("Salary:", salary)

experience = np.array([1, 2, 3, 5, 4])
print("Experience:", experience)

corr_matrix = np.corrcoef(salary, experience)
print("Correlation matrix:\n", corr_matrix)

rounded_corr = np.round(corr_matrix, 3)
print("Rounded correlation matrix:\n", rounded_corr)
```

OUR LOCATIONS

Output / Result

```
Salary: [20000 30000 40000 50000 60000]
Experience: [1 2 3 5 4]
Correlation matrix:
[[1.  0.9]
 [0.9 1.  ]]
Rounded correlation matrix:
[[1.  0.9]
 [0.9 1.  ]]
```

Explanation

The off-diagonal value shows the correlation between salary and experience; a value close to 1 means they tend to increase together.

Example 2: A near-zero correlation

```
import numpy as np

np.random.seed(4)
x = np.arange(10)
print("Array x:", x)

y = np.random.randint(1, 100, 10)
print("Array y:", y)

corr_matrix = np.corrcoef(x, y)
print("Correlation matrix:\n", corr_matrix)

rounded_corr = np.round(corr_matrix, 3)
print("Rounded correlation matrix:\n", rounded_corr)
```

Output / Result

```
Array x: [0 1 2 3 4 5 6 7 8 9]
Array y: [47 56 70  2 88 73 51 10 59 95]
Correlation matrix:
[[1.          0.17358256]
 [0.17358256  1.          ]]
Rounded correlation matrix:
[[1.    0.174]
 [0.174 1.    ]]
```

Explanation

When one variable is essentially unrelated to the other, the off-diagonal correlation value sits close to 0.

Example 3: Every array perfectly correlates with itself

```
import numpy as np

salary = np.array([20000, 30000, 40000, 50000, 60000])
print("Salary:", salary)

self_corr = np.corrcoef(salary, salary)
print("Correlation of salary with itself:\n", self_corr)
```

OUR LOCATIONS

Output / Result

```
Salary: [20000 30000 40000 50000 60000]
Correlation of salary with itself:
[[1. 1.]
 [1. 1.]]
```

Explanation

The diagonal of any correlation matrix is always 1, since a variable is always perfectly correlated with itself.

12. np.isin

Checks, element by element, whether values from one array also appear in another array.

Syntax

```
np.isin(a, test_values)
```

Example 1: Keeping only values from a target list

```
import numpy as np

a = np.array([15, 20, 35, 40, 55, 60, 75, 80])
print("Original array:", a)

items = [20, 40, 60, 80, 100]
print("Target list:", items)

mask = np.isin(a, items)
print("Membership mask:", mask)

filtered = a[mask]
print("Values found in target list:", filtered)
```

Output / Result

```
Original array: [15 20 35 40 55 60 75 80]
Target list: [20, 40, 60, 80, 100]
Membership mask: [False  True False  True False  True False  True]
Values found in target list: [20 40 60 80]
```

Explanation

np.isin builds a Boolean mask marking every element of a that also occurs in items, which can then be used to filter a directly.

Example 2: Excluding values with the negation

```
import numpy as np

a = np.array([15, 20, 35, 40, 55, 60, 75, 80])
print("Original array:", a)

items = [20, 40, 60, 80, 100]
mask = np.isin(a, items)
print("Membership mask:", mask)

negated_mask = ~mask
print("Negated mask:", negated_mask)

excluded = a[negated_mask]
print("Values NOT found in target list:", excluded)
```

Output / Result

```
Original array: [15 20 35 40 55 60 75 80]
Membership mask: [False True False True False True False True]
Negated mask: [ True False  True False  True False  True False]
Values NOT found in target list: [15 35 55 75]
```

Explanation

Prefixing the mask with ~ flips True and False, so this keeps every element that is NOT found in items.

Example 3: Using isin on a 2D array

```
import numpy as np

b = np.arange(12).reshape(3, 4)
print("Original 2D array:\n", b)

target = [0, 5, 10]
print("Target values:", target)

mask = np.isin(b, target)
print("Membership mask (2D):\n", mask)
```

Output / Result

```
Original 2D array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Target values: [0, 5, 10]
Membership mask (2D):
[[ True False False False]
 [False  True False False]
 [False False  True False]]
```

Explanation

isin works on arrays of any shape; the result is a same-shaped Boolean array rather than a flat list.

OUR LOCATIONS

13. np.flip

Reverses the order of array elements along the given axis while preserving the array's shape.

Syntax

```
np.flip(a, axis=None)
```

Example 1: Flipping a 1D array

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print("Original array:", a)

flipped = np.flip(a)
print("Flipped array:", flipped)
```

Output / Result

```
Original array: [10 20 30 40 50]
Flipped array: [50 40 30 20 10]
```

Explanation

Without an axis, flip reverses the entire flattened sequence of elements, similar to `a[::-1]`.

Example 2: Flipping a 2D array along columns

```
import numpy as np

b = np.arange(12).reshape(3, 4)
print("Original 2D array:\n", b)

flipped_cols = np.flip(b, axis=1)
print("Flipped along axis=1 (columns):\n", flipped_cols)
```

Output / Result

```
Original 2D array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Flipped along axis=1 (columns):
[[ 3  2  1  0]
 [ 7  6  5  4]
 [11 10  9  8]]
```

Explanation

`axis=1` reverses the order of columns within every row, while the row order itself stays the same.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 3: Flipping along rows instead

```
import numpy as np

b = np.arange(12).reshape(3, 4)
print("Original 2D array:\n", b)

flipped_rows = np.flip(b, axis=0)
print("Flipped along axis=0 (rows):\n", flipped_rows)
```

Output / Result

```
Original 2D array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Flipped along axis=0 (rows):
[[ 8  9 10 11]
 [ 4  5  6  7]
 [ 0  1  2  3]]
```

Explanation

axis=0 instead reverses the order of the rows, while each row's own contents stay in the same order.

14. np.put and np.delete

np.put replaces specific elements of an array in place; np.delete returns a new array with certain elements removed.

Syntax

```
np.put(a, indices, values)
np.delete(a, indices, axis=None)
```

Example 1: Replacing elements in place with np.put

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print("Original array:", a)

np.put(a, [0, 1], [110, 530])
print("Array after np.put (in place):", a)
```

Output / Result

```
Original array: [10 20 30 40 50]
Array after np.put (in place): [110 530 30 40 50]
```

Explanation

np.put treats the array as flattened and directly overwrites the values at the given positions - unlike most NumPy functions, it modifies a in place rather than returning a new array.

OUR LOCATIONS

Example 2: Deleting elements from a 1D array

```
import numpy as np

b = np.array([10, 20, 30, 40, 50])
print("Original array:", b)

after_delete = np.delete(b, [0, 2, 4])
print("Array after deleting indices [0, 2, 4]:", after_delete)
```

Output / Result

```
Original array: [10 20 30 40 50]
Array after deleting indices [0, 2, 4]: [20 40]
```

Explanation

np.delete returns a new array with the elements at the given positions removed, leaving the original array b untouched.

Example 3: Deleting a whole row from a 2D array

```
import numpy as np

c = np.arange(12).reshape(3, 4)
print("Original 2D array:\n", c)

after_delete = np.delete(c, 1, axis=0)
print("Array after deleting row index 1 (axis=0):\n", after_delete)
```

Output / Result

```
Original 2D array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Array after deleting row index 1 (axis=0):
[[ 0  1  2  3]
 [ 8  9 10 11]]
```

Explanation

Passing axis=0 removes an entire row (here, row index 1) instead of individual flattened elements.

15. Set functions

NumPy treats 1D arrays as mathematical sets, offering union, intersection, difference, and membership operations.

Syntax

```
np.union1d(a, b); np.intersect1d(a, b)
np.setdiff1d(a, b); np.setxor1d(a, b)
```

OUR LOCATIONS

Example 1: Union and intersection

```
import numpy as np

m = np.array([1, 2, 3, 4, 5])
print("Array m:", m)

n = np.array([3, 4, 5, 6, 7])
print("Array n:", n)

union = np.union1d(m, n)
print("Union:", union)

intersection = np.intersect1d(m, n)
print("Intersection:", intersection)
```

Output / Result

```
Array m: [1 2 3 4 5]
Array n: [3 4 5 6 7]
Union: [1 2 3 4 5 6 7]
Intersection: [3 4 5]
```

Explanation

Union keeps every distinct value from both arrays; intersection keeps only the values common to both.

Example 2: Difference and symmetric difference

```
import numpy as np

m = np.array([1, 2, 3, 4, 5])
print("Array m:", m)

n = np.array([3, 4, 5, 6, 7])
print("Array n:", n)

difference = np.setdiff1d(n, m)
print("n - m (values in n but not m):", difference)

sym_diff = np.setxor1d(m, n)
print("Symmetric difference:", sym_diff)
```

Output / Result

```
Array m: [1 2 3 4 5]
Array n: [3 4 5 6 7]
n - m (values in n but not m): [6 7]
Symmetric difference: [1 2 6 7]
```

Explanation

setdiff1d(n, m) keeps values in n that are not in m; setxor1d keeps values present in exactly one of the two arrays.

Example 3: Checking membership with in1d

```
import numpy as np

m = np.array([1, 2, 3, 4, 5])
print("Array m:", m)

mask = np.isin(m, [1])
print("Membership mask for value 1:", mask)

filtered = m[mask]
print("Filtered result:", filtered)
```

Output / Result

```
Array m: [1 2 3 4 5]
Membership mask for value 1: [ True False False False False]
Filtered result: [1]
```

Explanation

Filtering with a membership test (the modern equivalent of the older np.in1d) picks out only the elements of m that match a given value or list of values.

16. np.clip

Limits (clips) every value in an array to stay within a given minimum and maximum range.

Syntax

```
np.clip(a, a_min, a_max)
```

Example 1: Clipping to both a minimum and maximum

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

clipped = np.clip(a, a_min=25, a_max=75)
print("Clipped between 25 and 75:", clipped)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Clipped between 25 and 75: [25 25 57 73 25 25 25 75 42 25]
```

Explanation

Every value below 25 is raised to 25, every value above 75 is lowered to 75, and values already inside the range are left unchanged.

OUR LOCATIONS

Example 2: Clipping with only a lower bound

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

clipped_lower = np.clip(a, a_min=50, a_max=None)
print("Clipped with only a lower bound (50):", clipped_lower)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Clipped with only a lower bound (50): [50 50 57 73 50 50 50 75 50 50]
```

Explanation

Passing `a_max=None` leaves the upper end unrestricted, so this only raises values below 50 up to 50.

Example 3: Clipping with only an upper bound

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print("Original array:", a)

clipped_upper = np.clip(a, a_min=None, a_max=50)
print("Clipped with only an upper bound (50):", clipped_upper)
```

Output / Result

```
Original array: [25  4 57 73  1 22 20 75 42 11]
Clipped with only an upper bound (50): [25  4 50 50  1 22 20 50 42 11]
```

Explanation

Passing `a_min=None` leaves the lower end unrestricted, so this only lowers values above 50 down to 50.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune